

# Lecture 14: Shortest Path & Topological Sorting

## ★ Lecture objective

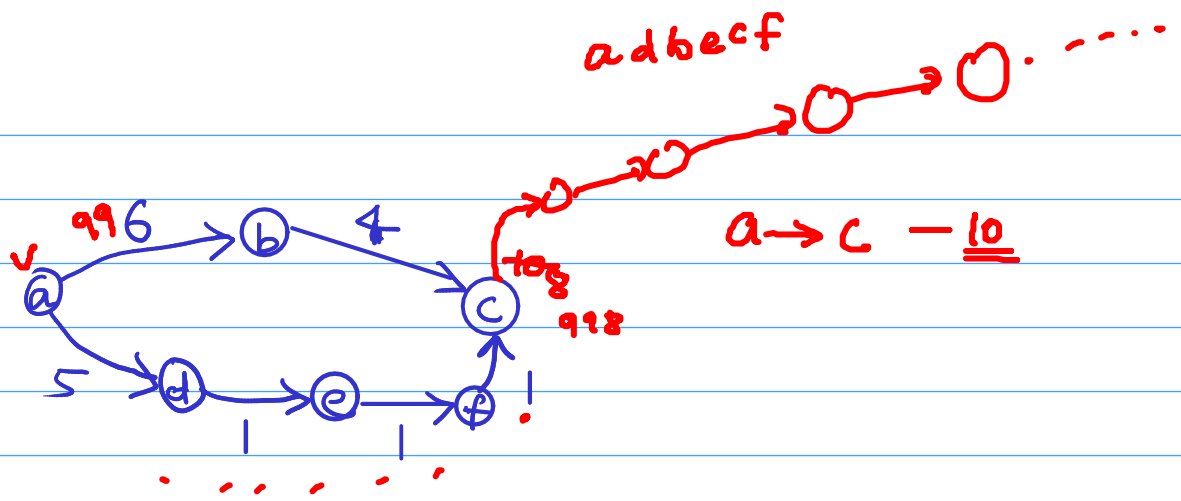
- To figure out efficient way to find shortest path from a given vertex to all other vertices.
- To understand topological ordering of DAG. And algorithm to find topological ordering of DAG using DFS.
- At last, we will discuss problem with solution discussed in class to identify if given graph is semiconnected or not.

## ★ Single Source shortest path

↳ Find shortest route from a given vertex to all other vertices.

↳ Can BFS be used?

↳ Yes, only if weights of the edges are same. WHY?



→ According to BFS, path from a to c has weight  $6+4=10$ . While shortest path from a to c is  $5+1+1+1=8$ .

→ You may argue that we can update the weight of path from a to c when c is reached via  $5+1+1+1$ . But then if there were other nodes already visited from c, then you will need to change all of them and so on. So, it is no longer BFS, and its complexity will increase.

↳ So, what else can we do?

→ There already is well known algorithm — Dijkstra's algo.

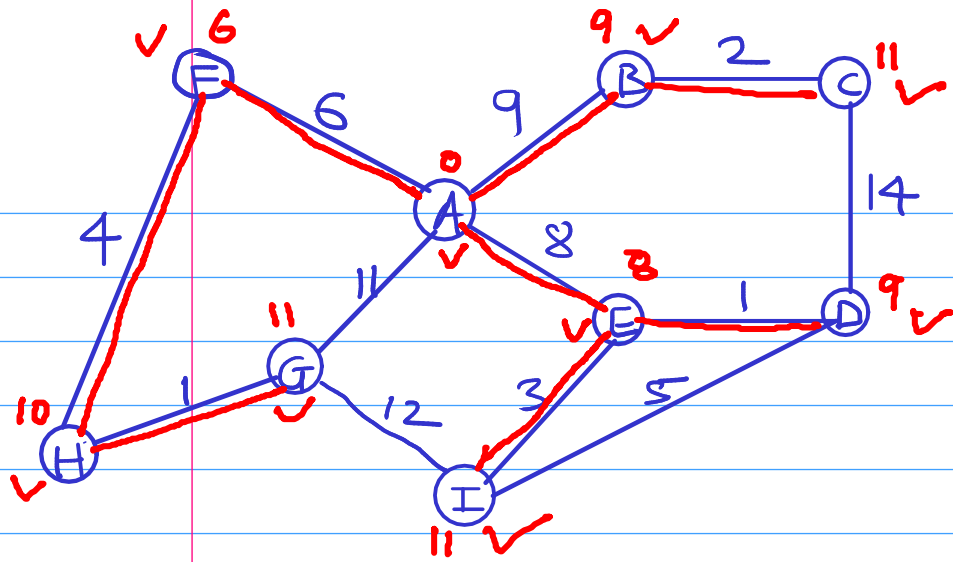
→ It is similar to Prim's algo.

→ But it chooses edges differently.

→ Lets look into example.

Note

→ Does not work if graph has (-)ve weights



How does Dijkstra's algo work?

① pick a source and marks it's distance 0.

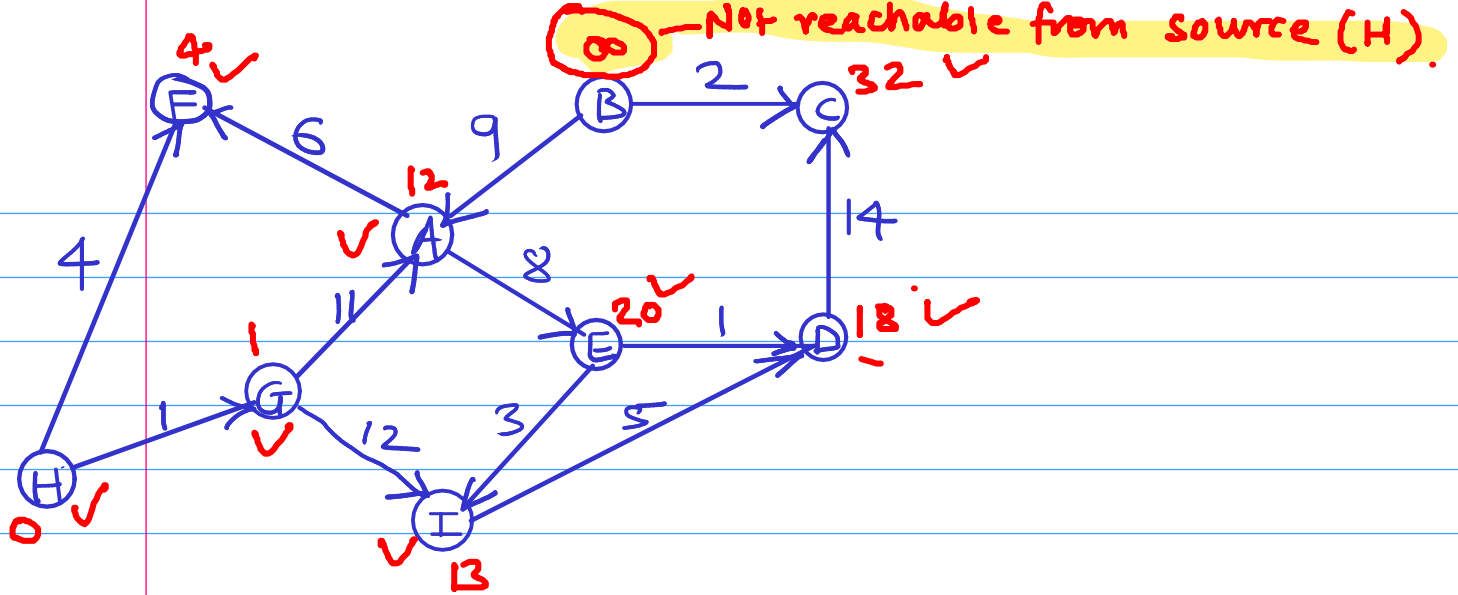
② Mark distance of all other vertices as max value ( $\infty$ )

③ Put all the vertices to  $S'$   
(no mark)

④ Repeat following still  $S'$  is empty  
④.1 pick vertex with smallest distance from  $S'$ . Move it to  $S$  (mark it)

④.2 Update distance of neighbours  
(if required)

→ It works for directed graph too.



How to implement?

<https://www.youtube.com/watch?v=IAXZGERcDf4>

$$V \rightarrow \max. E \quad \sim \underline{V^2}$$

→ What will be the complexity of it?  
 → Using heap, it can be reduced to  $O(E \log(E))$

sometimes referred as  $O(E \log(V^2))$   
 $O(2E \log(V))$   
 $O(E \log(V))$

→ Both are correct. why?

⇒ How to find all-pairs shortest path?

↳ V times execution of Dijkstra. once for each vertex.

↳ complexity?

$$\underline{V \times O(E * \log(E))}$$

$$\textcircled{\checkmark} \underline{O(V * E * \log(E))}$$

↳ Floyd-Warshall algo also gives, all-pairs shortest path.

Its complexity is  $O(V^3)$ .  
time

Ignore  
it for  
now!

~~so, in theory it's not better than running  
Dijkstra's algo V times.~~

~~But, in practice, Floyd-Warshall  
performs better.~~

~~true only  
for sparse graph~~

✓ <https://stackoverflow.com/questions/10779054/time-complexity-of-floyd-warshall-algorithm>

✓ <https://www.geeksforgeeks.org/comparison-dijkstras-floyd-warshall-algorithms/>

✓ [https://www.researchgate.net/post/Floyd-Warshall\\_algorithm\\_or\\_Dijkstras\\_Algorithm](https://www.researchgate.net/post/Floyd-Warshall_algorithm_or_Dijkstras_Algorithm)

→ Floyd-warshall algo will be covered in DAA subject.

→ what is path matrix?

! → can we use Dijkstra to generate path matrix?

A

A<sup>2</sup> i j

A<sup>3</sup>

(H.W.) ✓

## ★ Topological ordering (sorting)

→ For a given DAG, arrange (order) vertices in a sequence

$V_1, V_2, V_3, V_4, \dots, V_n$

Such that there does not exist a path from  $V_i$  to any vertex before it ( $V_1$  to  $V_{i-1}$ ).

→ In other words for each edge  $V_i \rightarrow V_j$

$V_i$  appears before  $V_j$  in the sequence

→ Is topological ordering possible if it is not DAG (if graph has cycle)?

AB

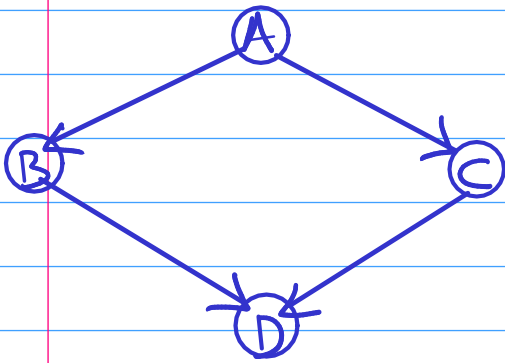


No. Why?

→ Topological sort for undirected graph is also not possible. As edges have no direction, there is no one order for an edge.

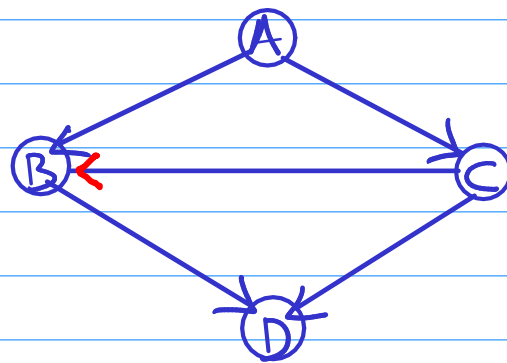
$a - b$   
 $b - a$  | same

which of these are valid topological sorts for given graph?



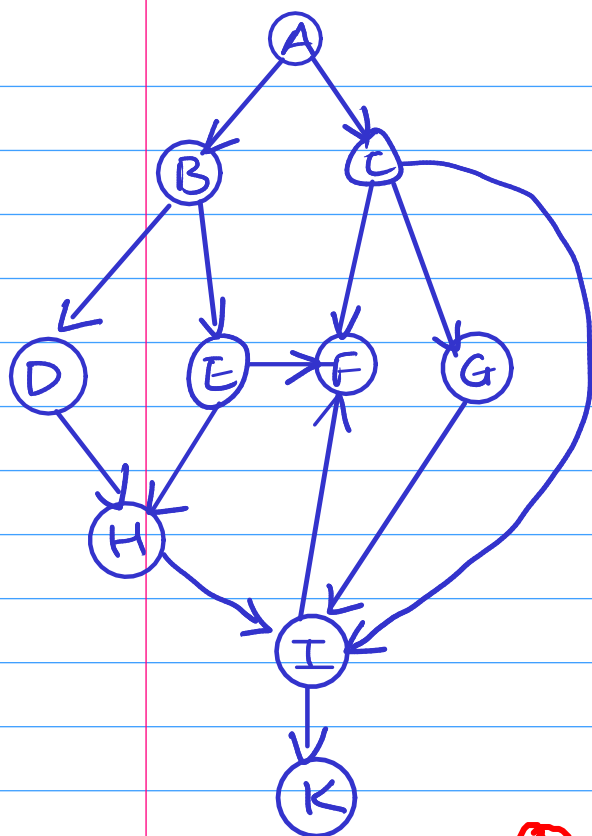
|      |   |     |   |
|------|---|-----|---|
| ABCD | ✓ | ] / | x |
| ACBD | ✓ |     | ✓ |
| BACD | x |     | x |
| BCAD | x |     | x |
| DBCA | x |     | x |

→ What if graph was as follow? which of above are still topological sorts?

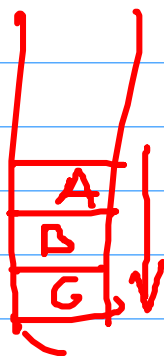
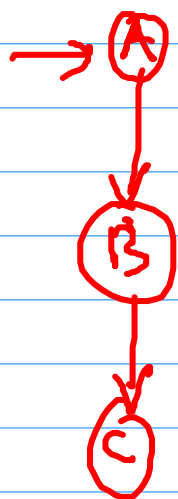


→ How can we use DFS for this? → Will see next.

→ Can BFS be used? Yes. Figure out yourself, how BFS can be used to come up with topological ordering for a given graph.

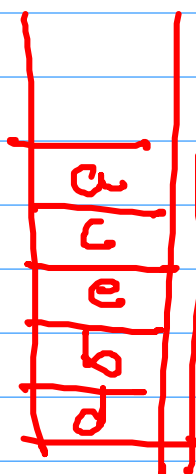
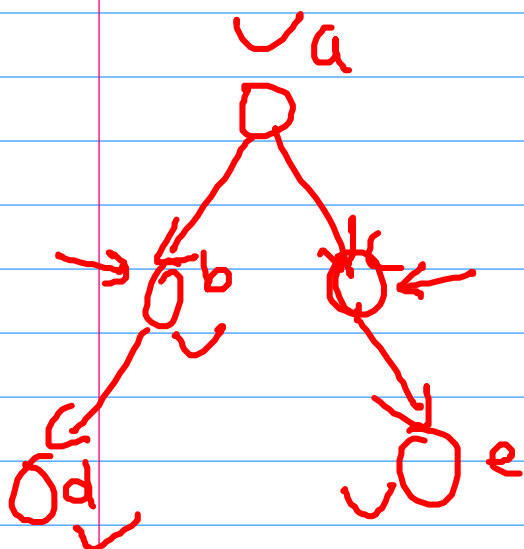


- ① Pick any <sup>unvisited</sup> vertex at random and apply DFS (recursive)
- ② while backtracking from a vertex, insert it into a separate stack.
- ③ If all vertices are not visited, goto Step ①.
- ④ Pop vertices from stack one by one and print them. It will produce topological sort.



ABC

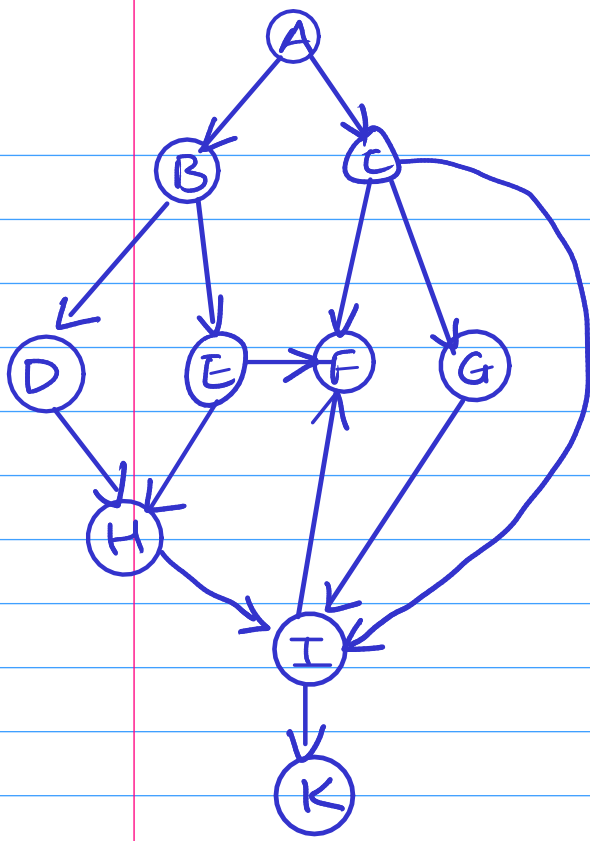
B



acebd

stack content  
 → d b e c a  
 → e c d b a



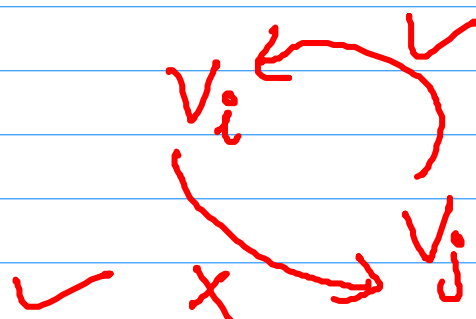


★ Issue with solution discussed in class to check if given digraph is semiconnected or not?

→ Do you remember the sol<sup>n</sup> discussed in class?

✓  
Semiconnected graph

|   | There exist a vertex from which all other vertices are <u>reachable</u> |
|---|-------------------------------------------------------------------------|
| T | T                                                                       |
| T | F                                                                       |
| F | T                                                                       |
| F | F                                                                       |

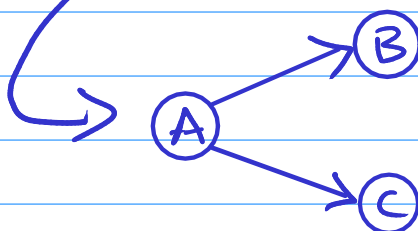


Not possible. We argued this  
we missed this ☹️

$$V \times O(V + E)$$

$$O(V^2 + VE)$$

e.g.



→ All vertices are reachable from A.  
→ But no path from B to C or C to B. So, it is not semiconnected.

|     |   |   |   |     |   |
|-----|---|---|---|-----|---|
|     | 1 | 2 | 3 | ... | v |
| 1   | 1 | 0 | 1 | 1   |   |
| 2   | 0 |   |   |     |   |
| ... |   |   |   |     |   |
| v   |   |   |   |     |   |

✓

→ Link to better sol<sup>n</sup>

→ what is the solution then?

① Path matrix can be used as discussed above.

② There is better solution using strongly connected components and topological ordering.

<https://stackoverflow.com/questions/30642383/determine-if-a-graph-is-semi-connected-or-not>

NOTE: Notes with this color have been added after lecture.